

Salesforce Developer Bridge Program – Day 1 Documentation

Name: G. Bhanu Durga Ram Prasad

Technology: Salesforce Development

Platform: Salesforce Trailhead Playground

Task 1 – Data Model Design

Objective

The objective of this task was to design and implement a simple Hospital Management System data model in Salesforce. The data model consists of three custom objects: **Patient**, **Doctor**, and **Appointment**. Relationships were established between these objects to represent real-world hospital operations. This activity helped in understanding Salesforce data modeling concepts, custom objects, custom fields, and object relationships. The training specifically required creating custom objects and relationship fields in a Trailhead Playground.

Objects Created

- Patient__c
- Doctor__c
- Appointment__c

Relationship Used

- Appointment → Patient (Lookup Relationship)
- Appointment → Doctor (Lookup Relationship)

Lookup relationships were chosen because appointments can exist independently without deleting related patient or doctor records.

Activities Performed

- Created three custom objects in Salesforce.
- Added custom fields such as Name, Age, Gender, Appointment Date, and Status.
- Configured Lookup Relationships between Appointment, Patient, and Doctor.
- Verified the data model using Salesforce Object Manager.

Developer Edition

Search Setup

Setup Home Object Manager

SETUP > OBJECT MANAGER

patient

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Details

Description

API Name

patient__c

Custom

✓

Singular Label

patient

Plural Label

patients

Enable Reports

Track Activities

Track Field History

Deployment Status

Deployed

Help Settings

Standard salesforce.com Help Window

EditDelete

SETUP > OBJECT MANAGER

Appointment

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Details

Description

API Name

Appointment__c

Custom

✓

Singular Label

Appointment

Plural Label

Appointments

Enable Reports

Track Activities

Track Field History

Deployment Status

Deployed

Help Settings

Standard salesforce.com Help Window

EditDelete

SETUP > OBJECT MANAGER

Doctor

Details

Fields & Relationships

Page Layouts

Lightning Record Pages

Buttons, Links, and Actions

Compact Layouts

Field Sets

Object Limits

Record Types

Related Lookup Filters

Restriction Rules

Scoping Rules

Details

Description

API Name

Doctor__c

Custom

✓

Singular Label

Doctor

Plural Label

Doctors

Enable Reports

Track Activities

Track Field History

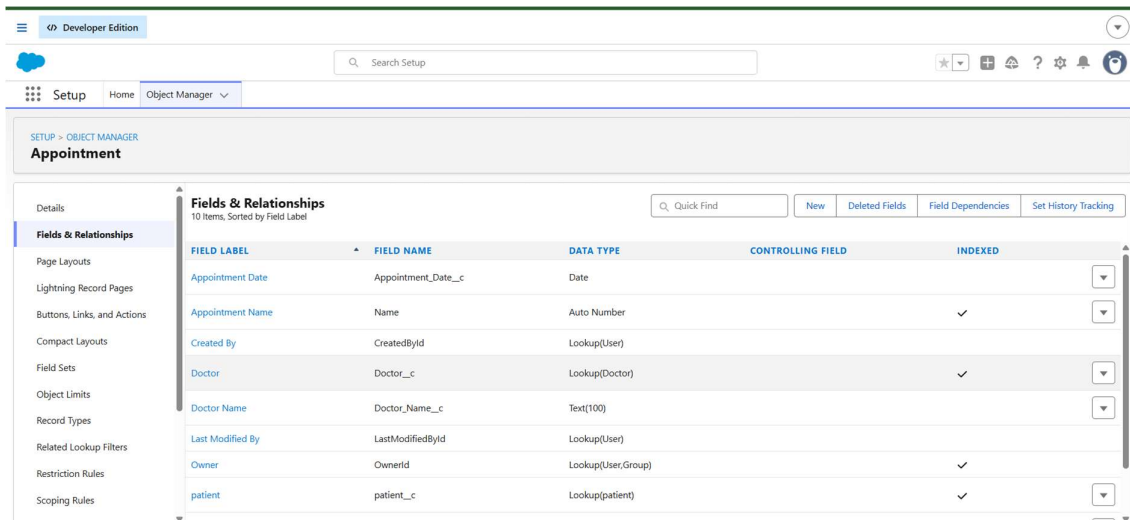
Deployment Status

Deployed

Help Settings

Standard salesforce.com Help Window

EditDelete



Outcome

The custom objects and their relationships were successfully created, forming a simple Hospital Management System that can store patient, doctor, and appointment information.

Task 2 – Apex Basics

Objective

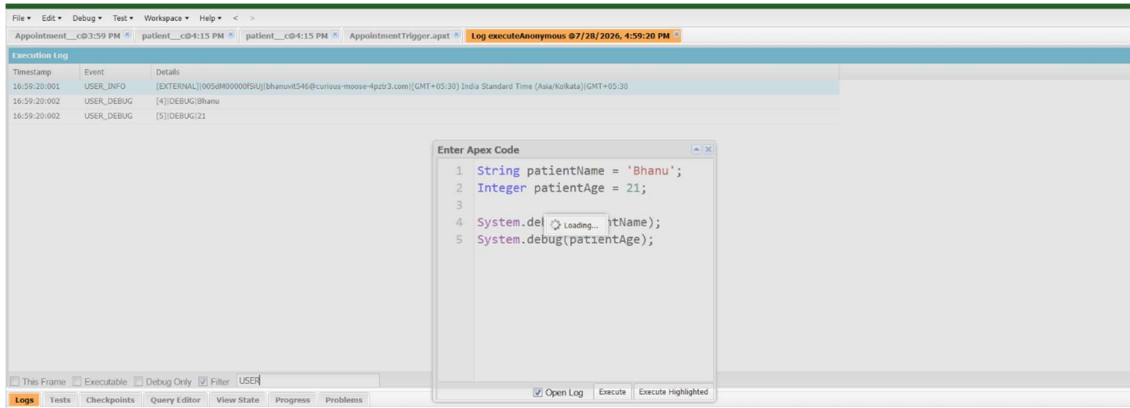
The objective of this task was to understand the fundamentals of Apex programming and become familiar with the Salesforce Developer Console. The task involved learning Apex syntax, executing Apex code, and analyzing execution results using debug logs. The training recommended practicing Apex Basics through Trailhead and the Developer Console.

Activities Performed

- Studied Apex programming fundamentals.
- Opened the Salesforce Developer Console.
- Practiced writing basic Apex statements.
- Executed Apex code using the Execute Anonymous Window.
- Verified successful execution through debug logs.

Learning Outcome

This activity provided practical knowledge of Salesforce server-side programming using Apex and introduced the Developer Console for writing and testing code.



Task 3 – SOQL Practice

Objective

The objective of this task was to learn Salesforce Object Query Language (SOQL) by retrieving records from custom objects using different query techniques. The task included writing queries with filtering, sorting, aggregate functions, and comparison operators.

Query 1 – Retrieve Patient Records

SELECT Name, Age__c, Gender__c

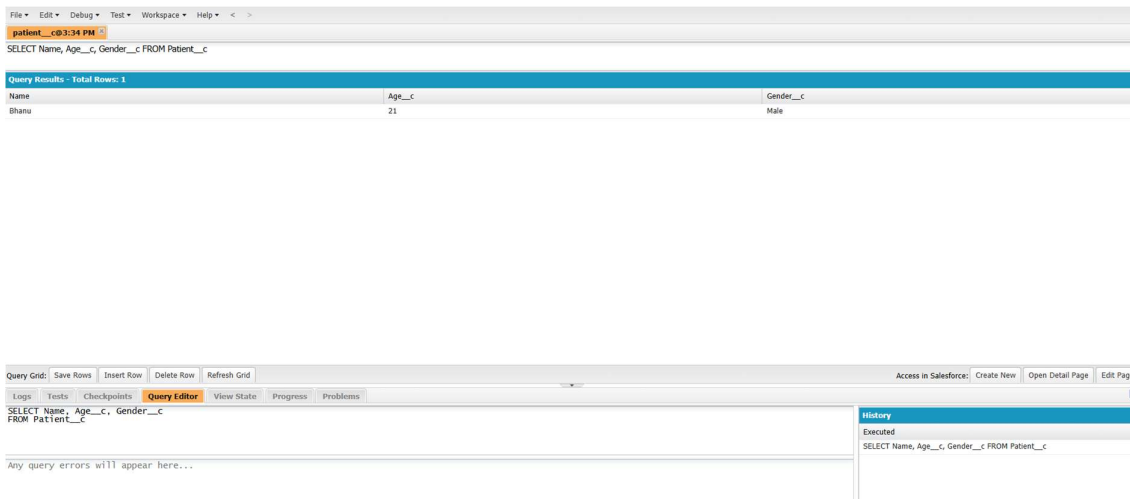
FROM Patient__c

Description

This query retrieves the patient's Name, Age, and Gender from the Patient object.

Observation

Initially, no records were available. After inserting patient records, the query successfully displayed the stored information.



Query 2 – ORDER BY and LIMIT

```
SELECT Name, Age__c  
FROM Patient__c  
ORDER BY Age__c DESC  
LIMIT 5
```

Description

This query sorts patient records by age in descending order and displays only the first five records.

Observation

The query returned patient records successfully, sorted from the highest age to the lowest.



Query Results - Total Rows: 3

Name	Age__c
Crish	25
Pavan	22
Bhanu	21



Query Editor

```
SELECT Name, Age__c  
FROM Patient__c  
ORDER BY Age__c DESC  
LIMIT 5
```

Any query errors will appear here...

Execute ☐ Use Tooling API

History

- Executed
- SELECT Name, Age__c, Gender__c FROM Patient__c
- SELECT Name, Age__c FROM Patient__c ORDER BY Age__c DESC LIMIT 5

Query 3 – Aggregate Query

```
SELECT COUNT(Id)  
FROM Patient__c
```

Description

This aggregate query counts the total number of patient records available in the Patient object.

Observation

The query successfully displayed the total number of patient records stored in Salesforce.

File • Edit • Debug • Test • Workspace • Help • < • >

Appointment__c@3:59 PM • patient__c@4:15 PM

SELECT COUNT(Id) FROM Patient__c

Query Results - Total Rows: 1

count(Id)
3

Query Grid: Save Rows Insert Row Delete Row Refresh Grid

Access in Salesforce: Create New Open Detail Page Edit Page

Logs Tests Checkpoints **Query Editor** View State Progress Problems

SELECT COUNT(Id)
FROM Patient__c

Any query errors will appear here...

Execute ☐ Use Tooling API

History

Executed

SELECT Name, Patient__cName FROM Appointment__c

SELECT COUNT(Id) FROM Patient__c

Query 4 – Comparison Operator

SELECT Name, Age__c

FROM Patient__c

WHERE Age__c >= 21

Description

This query retrieves all patient records where the age is greater than or equal to 21.

Observation

The comparison operator successfully filtered and displayed only the matching patient records.

Query Results - Total Rows: 3

Name	Age__c
Girish	25
Bhanu	21
Pavan	22

Query Grid: Save Rows Insert Row Delete Row Refresh Grid

Access in Salesforce: Create New Open Detail Page Edit Page

Logs Tests Checkpoints **Query Editor** View State Progress Problems

SELECT Name, Age__c
FROM Patient__c
WHERE Age__c >= 21

Any query errors will appear here...

Execute ☐ Use Tooling API

History

Executed

SELECT Name, Patient__cName FROM Appointment__c

SELECT COUNT(Id) FROM Patient__c

SELECT Name, Age__c FROM Patient__c WHERE Age__c >= 21

Outcome

All SOQL queries executed successfully without syntax errors. Through this activity, I gained practical knowledge of retrieving, filtering, sorting, and aggregating data using SOQL.

Task 4 – Apex Trigger

Objective

The objective of this task was to create an Apex Trigger that validates appointment records before they are inserted into Salesforce. The trigger prevents users from creating appointments whose status is **Cancelled**, thereby enforcing a business rule. The training required writing a trigger and validating record creation before insertion.

Trigger Code

```
trigger AppointmentTrigger on Appointment__c (before insert) {

    for (Appointment__c app : Trigger.new) {

        if (app.Status__c == 'Cancelled') {

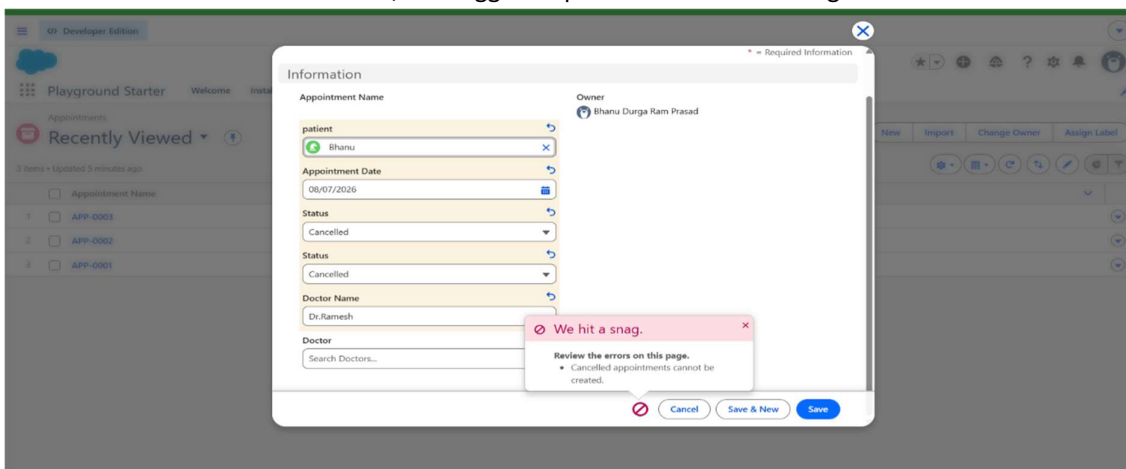
            app.addError('Cancelled appointments cannot be created.');
        }

    }

}
```

Working of the Trigger

1. The trigger executes before an Appointment record is inserted.
2. It loops through all new appointment records using Trigger.new.
3. It checks the value of the Status__c field.
4. If the status is **Cancelled**, the trigger stops the record from being saved.



5. Salesforce displays the message "**Cancelled appointments cannot be created.**"

Task 5 – Build a Lightning Web Component (LWC)

Objective

Develop a Lightning Web Component (LWC) that displays records from the custom Patient object using the @wire service. The component should retrieve real-time data from Salesforce and display it on a Lightning Home Page.

Implementation

- Created an Apex controller (PatientController) to fetch Patient records from the custom object.
- Developed an LWC named patientList.
- Used the @wire decorator to call the Apex method and retrieve patient data.
- Displayed the retrieved records in a tabular format with the following fields:
 - Patient Name
 - Age
 - Gender
 - Blood Group

11

- Deployed the component to the Salesforce Trailhead Playground.
- Created a Hospital Home Lightning Page using Lightning App Builder.
- Added the patientList component to the Home Page.
- Saved and activated the page as the Home Page.

Result

The Lightning Web Component successfully displays real Patient records from the custom Patient object. The component was added to the Hospital Home page through Lightning App Builder, and after activation, the records are displayed correctly without any errors, satisfying the expected outcome of Task 5.

Developer Edition

Search Setup

Setup Home Object Manager

Quick Find

Setup Home

Salesforce Go

Commerce Setup Assistant

Hyperforce Assistant

Release Updates

Salesforce Mobile App

Lightning Usage

AgentExchange Offers

Sales Cloud Everywhere

ADMINISTRATION

Users

Data

Email

PLATFORM TOOLS

Apps

Feature Settings

SETUP

Lightning App Builder

The Lightning App Builder provides an easy to use graphical interface for creating custom Lightning pages for Salesforce Lightning Experience and mobile app. Lightning pages are built using Lightning components—compact, configurable, and reusable elements that you can drag and drop into regions of the page in the Lightning App Builder.

View: All Create New View

Lightning Pages

New

Action	Label	Name	Namespace Prefix	Description	Type	Created By	Last Modified By
Edit	Clone	Del	hospital_home	hospital_home	Home Page	BSam, 29/07/2026, 4:14 pm	BSam, 29/07/2026, 4:14 pm

Developer Edition

Lightning App Builder Pages hospital_home

Desktop Shrink To View

Activation... Save

Page > helloWorld

Set Component Visibility

Filters

Add Filter

Components

Standard (2)

App Launcher

Chatter Publisher

Custom (1)

helloWorld

Custom - Managed (0)

Get more on AppExchange

JavaScript SDK

Patient Records

Native (Static) App: 21

Native (Graph) App: 23

Native (Page) App: 22

EXPLORER

helloWorld.html PatientController.cls PatientController.cls-meta.xml JS helloWorld.js helloWorld.js-meta.xml

OPEN EDITORS

HOSPITALMANAGEMENT

stx

vscode

config

force-app/main/default

applications

aura

classes

PatientController.cls

PatientController.cls-meta.xml

contentassets

flexipages

layouts

lwc

helloWorld

tests

gitignore

helloWorld.html

JS helloWorld.js

helloWorld.js-meta.xml

jsconfig.json

objects

permissionsets

staticresources

tabs

triggers

manifest

package.xml

scripts

forcedignore

gitignore

prettierrignore

OUTLINE

TIMELINE

force-app/main/default/lwc/helloWorld/helloWorld.js-meta.xml > XML > LightningComponentBundle

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <lightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
3   <apiVersion>64.0</apiVersion>
4   <isExposed>true</isExposed>
5   <targets>
6     <target>lightning_AppPage</target>
7     <target>lightning_HomePage</target>
8     <target>lightning_RecordPage</target>
9   </targets>
10 </lightningComponentBundle>
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

Deployed Source

State	Name	Type	Path
Created	helloWorld	LightningComponentBundle	force-app/main/default/lwc/helloWorld/helloWorld.html
Created	helloWorld	LightningComponentBundle	force-app/main/default/lwc/helloWorld/helloWorld.js
Created	helloWorld	LightningComponentBundle	force-app/main/default/lwc/helloWorld/helloWorld.js-meta.xml

DX Com... powershell powershell powershell powershell powershell powershell

